SE-3003 WEB ENGINEERING

Assignment – 01 – Total Marks 200

Due Date: [redacted] 19 April 2026 - Sunday (11:59 PM)

Instructions (Must Read)

- This is an individual assignment
- For timely completion of this assignment, start as soon as possible.
- All late assignments will be marked zero.
- Please do not remove the project from your laptops. It will be required for the viva.
- Use of AI tools is strongly discouraged.
- Copying from other students will awarded **zero marks in the whole assignment category**.
- Students who will not use AI will receive **+10 bonus marks**.
- If AI is used but the student cannot explain their own code, **no marks will be awarded**.
- If AI-generated code is used but the student understands and can explain it, **marks will be given based on understanding during viva**.
- It is highly recommended that AI should only be used for **learning and understanding concepts**, not for completing the assignment to ensure maximum marks in this assignment.
- Be prepared for **code walkthroughs, live modifications** and **oral verification.**
- **Submission Naming:** Submit a .zip file and rename as **SECTION_RollNo_A1**. Not following naming convention will result in 5% marks deduction.

Frontend Admin Dashboard Web Application

1. Objective

Students are required to design and develop a **multi-page admin dashboard web application** using:

- HTML (structure)
- CSS (layout, responsiveness, styling)
- JavaScript (logic, interactivity, data handling)

The application must simulate real-world functionality using **mock data and browser storage (localStorage)**.

2. General Rules and Constraints

1. No backend technologies are allowed (no Node.js, PHP, databases, etc.).
2. All data must be handled using:
    - JavaScript arrays (mock data)
    - localStorage (for persistence)
3. All pages must be connected and navigable.
4. Code must be modular, readable, and properly structured.
5. Responsive design is mandatory.
6. All features must function without page reloads where possible.

3. Authentication Workaround (Mandatory)

Since there is no backend, authentication must be simulated using **localStorage**.

1. Signup (User Creation)

- User enters email and password.
- Data is stored in **localStorage** (not sent to a server).
- Save as an object: { email, password }.

Summary: Browser acts as a temporary database.

2. Login (User Verification)

- Retrieve stored user data from **localStorage**.
- Compare entered email & password with stored values.

If match:

- Mark user as logged in (e.g., loggedIn = true in localStorage).
- Redirect to dashboard.

If not:

- Show error: *"Invalid email or password"*.

3. Session Simulation

- Store a flag (e.g., loggedIn) in **localStorage** after login.
- Use it to track login state.

Summary: localStorage acts as a fake session.

4. Route Protection

- On every protected page:
    - Check loggedIn in localStorage.

If not present:

- Redirect to login page.

If present:

- Allow access.

5. Limitations (Important)

- Not secure
- Data can be viewed/edited via browser dev tools
- Only for learning/demo purposes

4. Pages and Detailed Requirements

4.1 Landing Page (10 Marks)

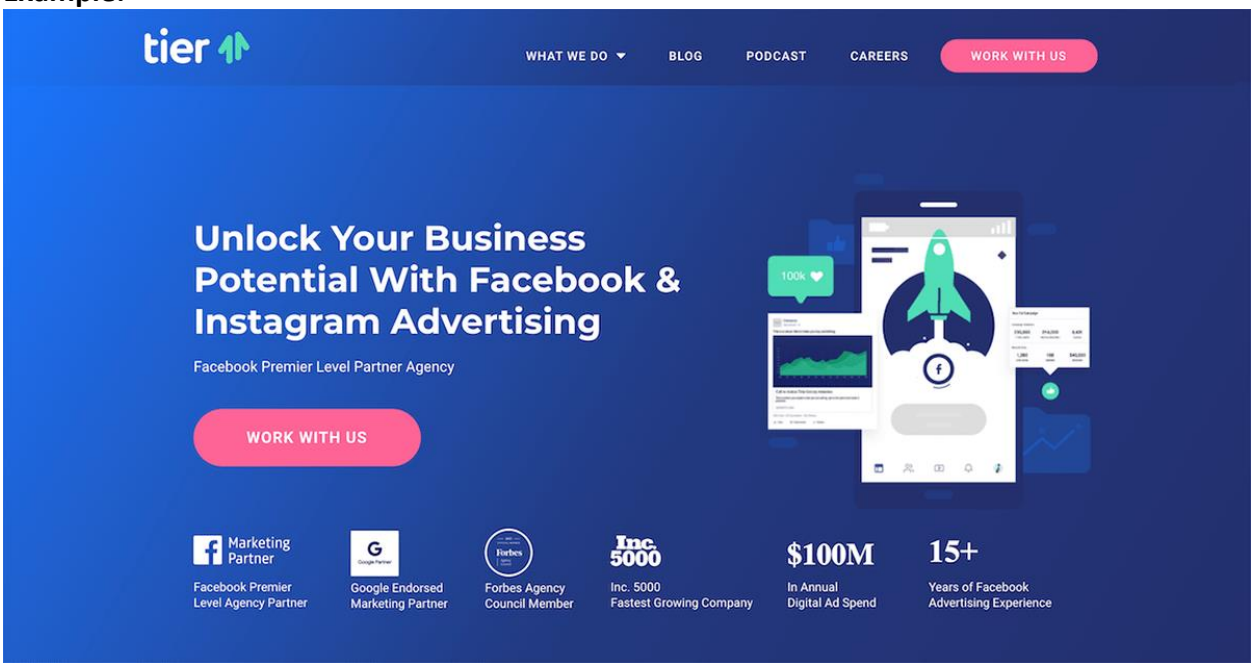
Requirements:

- Hero section:
  - Application name
  - Description
  - Buttons: Login, Signup
- Features section:
  - Minimum 3 feature cards
- Footer:
  - Basic links or information

Functional Tasks:

Task	Marks
Layout structure (HTML)	2
Styling and responsiveness	3
Navigation buttons working	3
Section organization	2

Example:



4.2 Login Page (10 Marks)

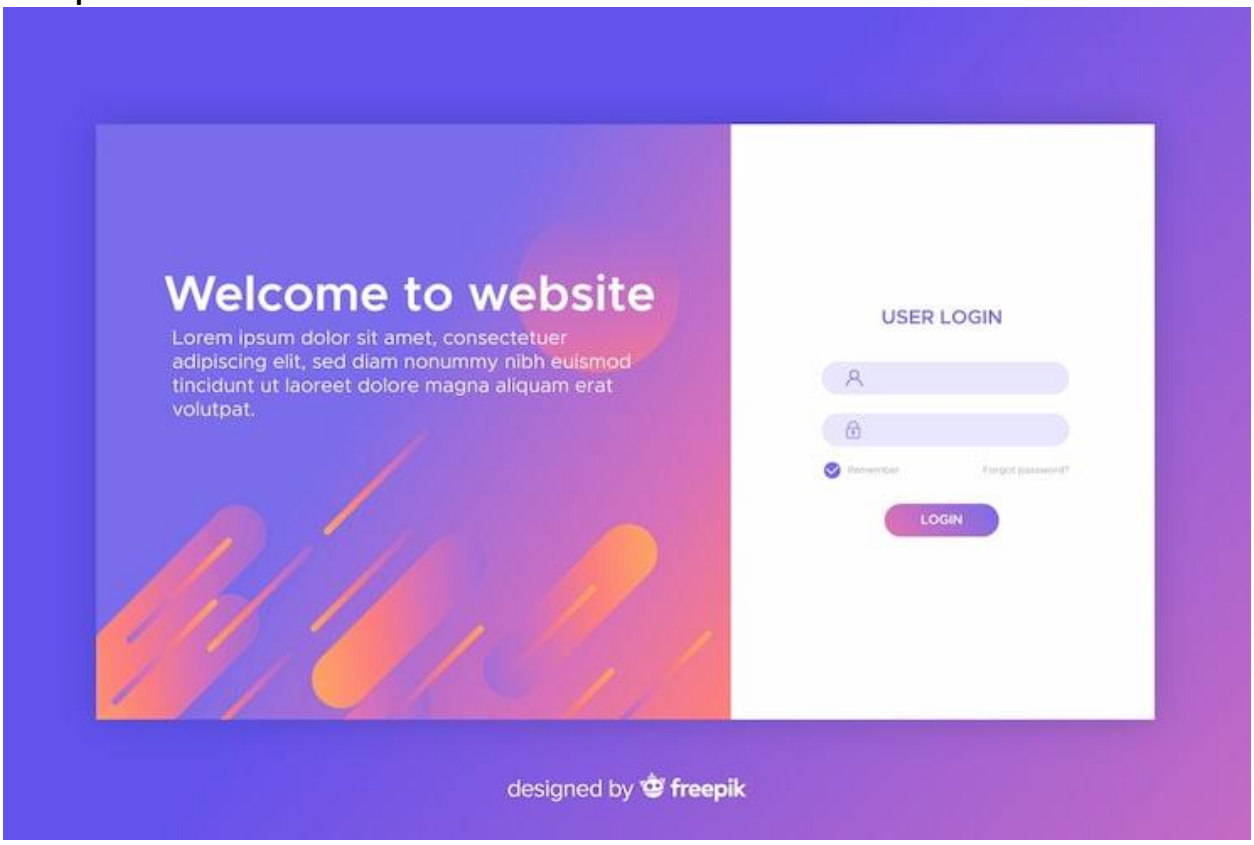
Requirements:

- Input fields:
  - Email
  - Password
- Validation:
  - Empty fields check
- Error messages

Functional Tasks:

Task	Marks
Form structure	2
Input validation	3
Login logic using localStorage	3
Error handling	2

Example:



4.3 Signup Page (10 Marks)

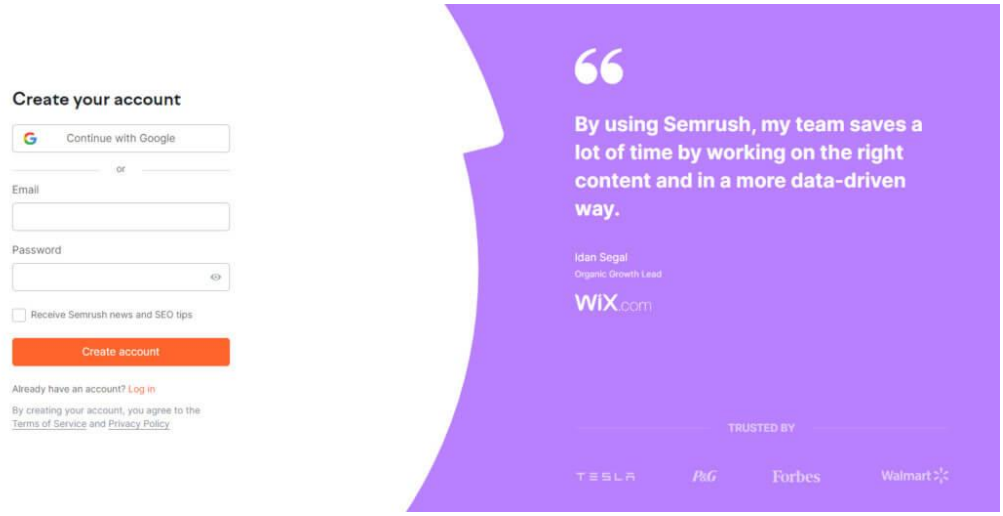
Requirements:

- Input fields:
  - Email
  - Password
  - Confirm Password
- Validation:
  - Password match
  - Required fields

Functional Tasks:

Task	Marks
Form structure	2
Validation logic	3
Saving user in localStorage	3
User feedback messages	2

Example:



4.4 Dashboard Page (15 Marks)

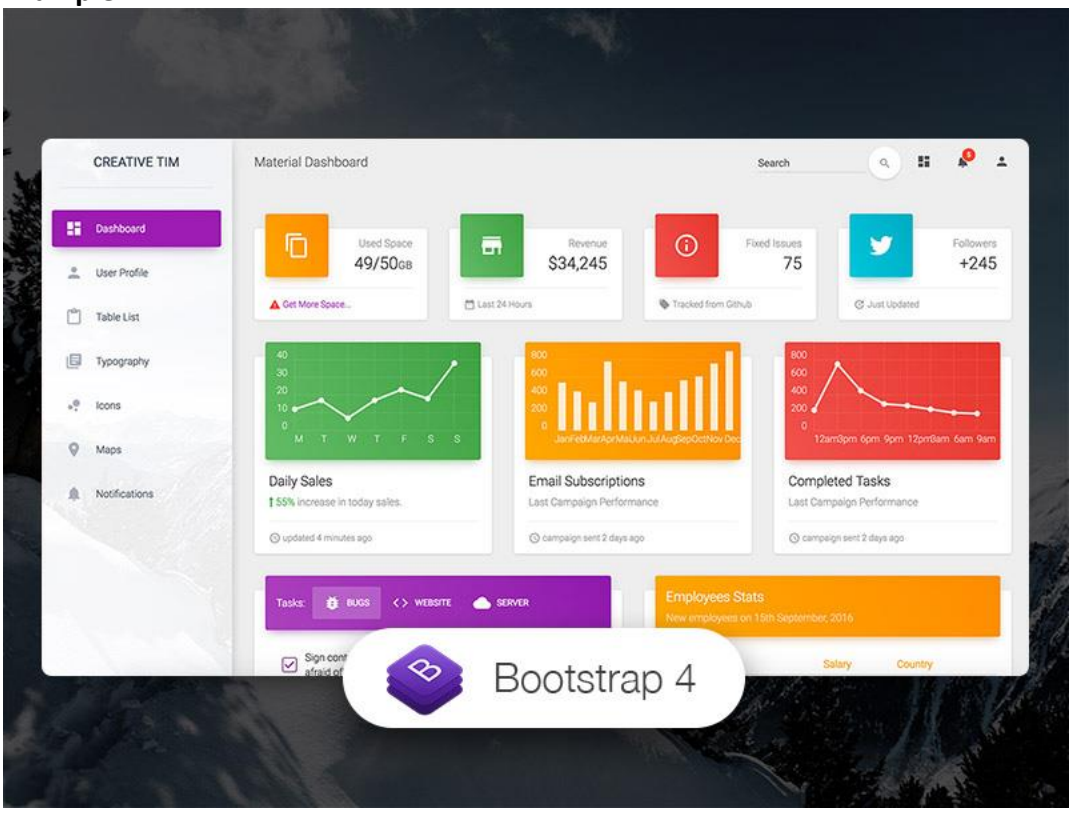
Requirements:

- Sidebar navigation
- Header (user info + logout button)
- Summary cards:
  - Total users
  - Total orders
  - Revenue
- Chart (mock data)

Functional Tasks:

Task	Marks
Layout (sidebar + header)	4
Cards with dynamic data	4
Chart implementation	4
Data rendering logic	3

Example:



4.5 Users Page (15 Marks)

Requirements:

- Table: Name, Email, Role, Status

Functional Features:

- Add user
- Edit user
- Delete user
- Search/filter users

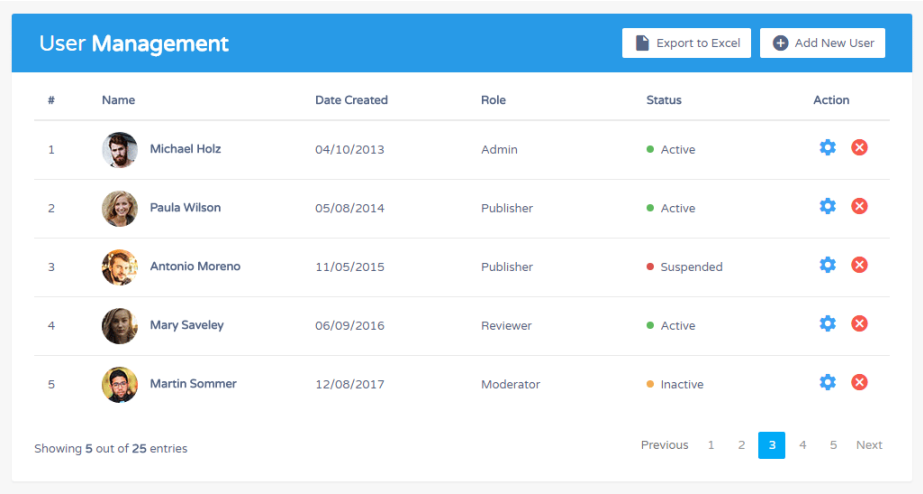
Data Handling:

- Use array of objects
- Store in localStorage

Functional Tasks:

Task	Marks
Table structure	3
Add user functionality	3
Edit user functionality	3
Delete user functionality	2
Search/filter logic	2
Data persistence	2

Example:



4.6 Products (15 Marks)

Requirements:

- Table: Name, Price, Status, Picture, Quantity

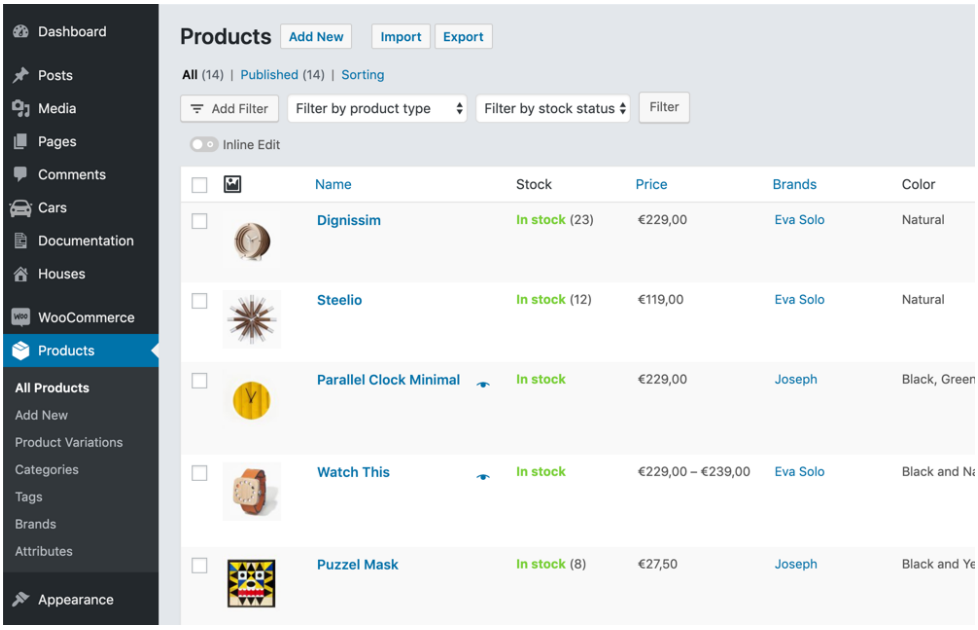
Functional Features:

- Add product/order
- Update logic
- Delete item
- Filtering and Sorting

Functional Tasks:

Task	Marks
Table layout	3
Add functionality	3
Sorting and Filtering	3
Delete functionality	2
Update logic	2
Data storage logic	2

Example:



4.7 Analytics Page (10 Marks)

Requirements:

- At least 2 charts: (Bar, Pie or line chart)

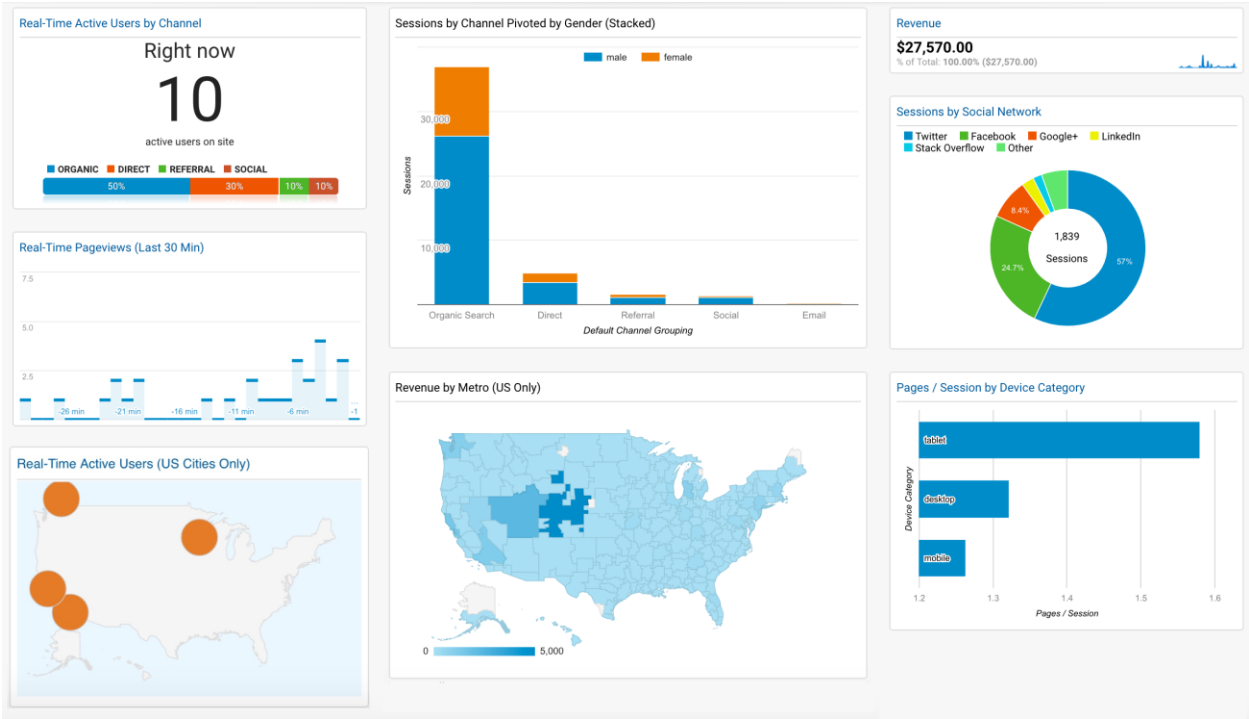
Data:

- Mock data only

Functional Tasks:

Task	Marks
Chart rendering	4
Data representation	3
Layout and styling	3

Example:



4.8 Notifications Page (5 Marks)

Requirements:

- List of notifications
- Example:
    - New user registered
    - Order placed

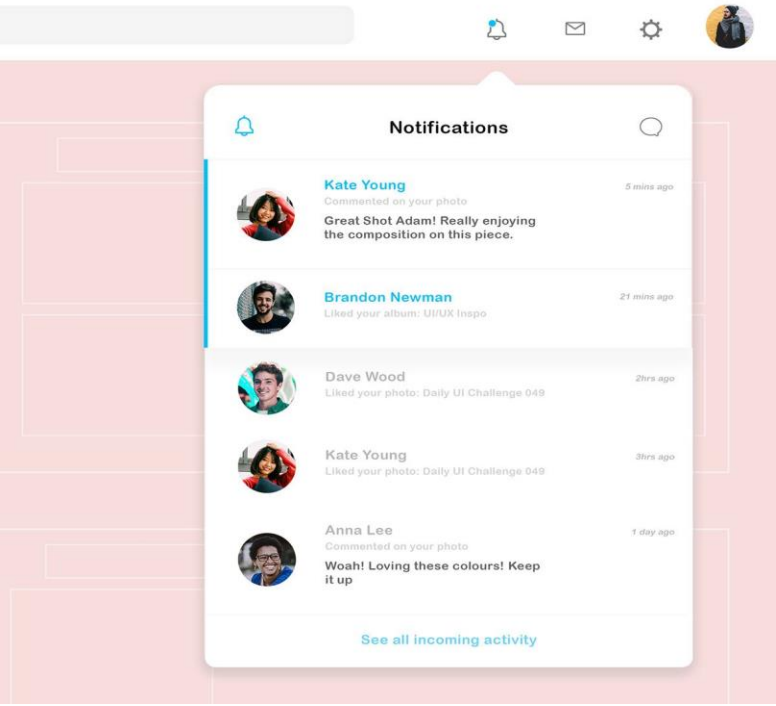
Functional Features:

- Mark as read
- Delete notification

Functional Tasks:

Task	Marks
List display	2
Mark as read	1
Delete functionality	1
Storage handling	1

Example:



4.9 Settings Page (5 Marks)

Requirements:

- Profile form:
  - Name
  - Email
- Password change (mock)
- Dark mode toggle

Functional Tasks:

Task	Marks
Form structure	2
Save settings in localStorage	2
Dark mode toggle	1

4.10 Logout Functionality (5 Marks)

Requirements:

- Clear session
- Redirect to login page

Functional Tasks:

Task	Marks
Logout logic	3
Redirect behavior	2

5. Global Features (50 Marks)

Task	Marks
Sidebar navigation across all pages	25
Simulated Authentication	25
Total Marks	**50**

6. Local Features (100 Marks)

Section	Marks
Landing Page	10
Login Page	10
Signup Page	10
Dashboard	15
Users	15
Products/Orders	15
Analytics	10
Notifications	5
Settings	5
Logout	5
Total	**100**

7. Expected Learning Outcomes

By completing this assignment, students will demonstrate:

- Understanding of multi-page frontend architecture
- Ability to manage state using localStorage
- DOM manipulation and event handling
- Implementation of CRUD operations
- Responsive UI design
- Simulation of real-world application behavior without backend

8. Rubrics

Section	Marks
Local Features	100
Global Features	50
Responsiveness	10
Design Consistency	10
Code Quality and Structure	10
Aesthetics	10
Deployment (Vercel, netliify, github pages etc)	10
Total Marks	**200**

9. Common Queries

Q: Can we use Tailwind? **(Yes)**
Q: Can we use libraries/frameworks like bootstrap? **(Yes)**
Q: Is this assignment time taking? **(Yes)**
Q: Should I start the assignment from day one? **(Yes)**

For more queries, you can comment in the GCR comment section.

Good Luck ☺